

PREDICTING STARGAZERS IN OPEN SOURCE PROJECTS

Final Project Presentation

GROUP 7

COURSE:

1TD076 - Data Engineering II

PRESENTED BY:

Bradley Deku
Ioannis Peridis
Kanza Raisi
Sotiris Oikonomou
William Min

PRESENTED ON:

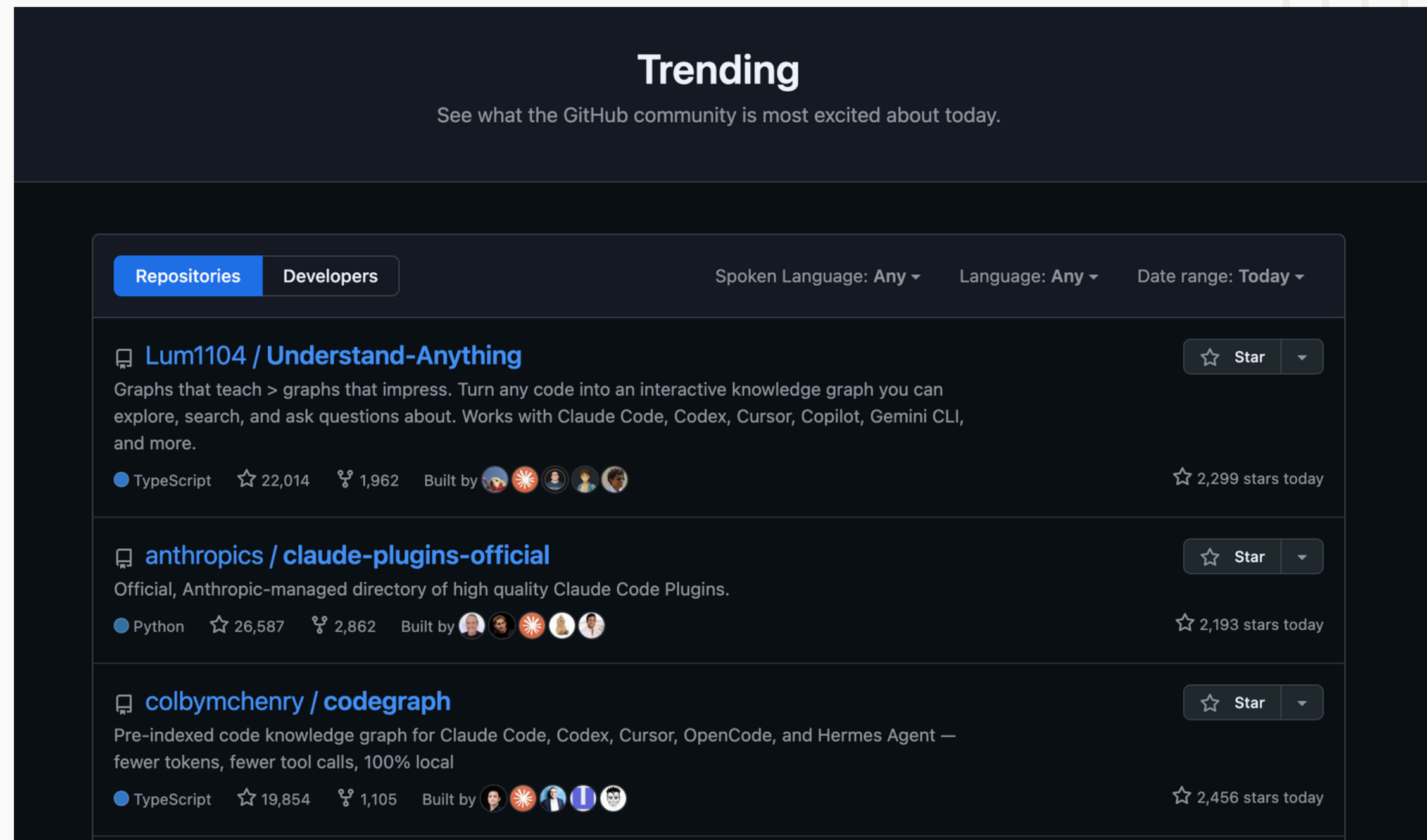
25th May 2026

Agenda

03	Motivation
05	System Architecture
06	Data Collection
07	Feature Engineering & Model Training
08	Deployment
09	Stargazer Application
11	Results

Motivation

Stars on GitHub are community-facing measure of project popularity



Motivation

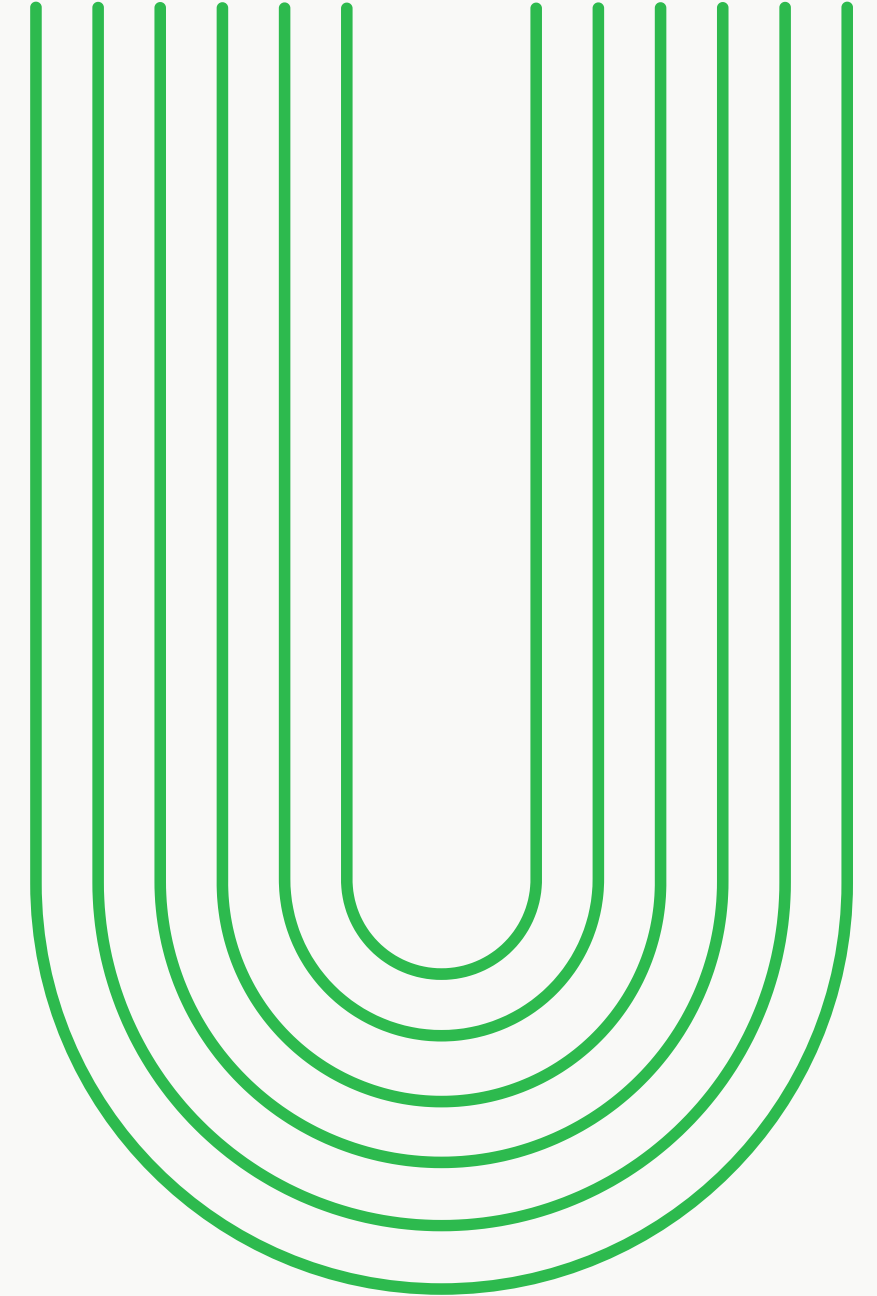
Stars on GitHub are community-facing measure of project popularity

Can we predict them from observable repository properties?

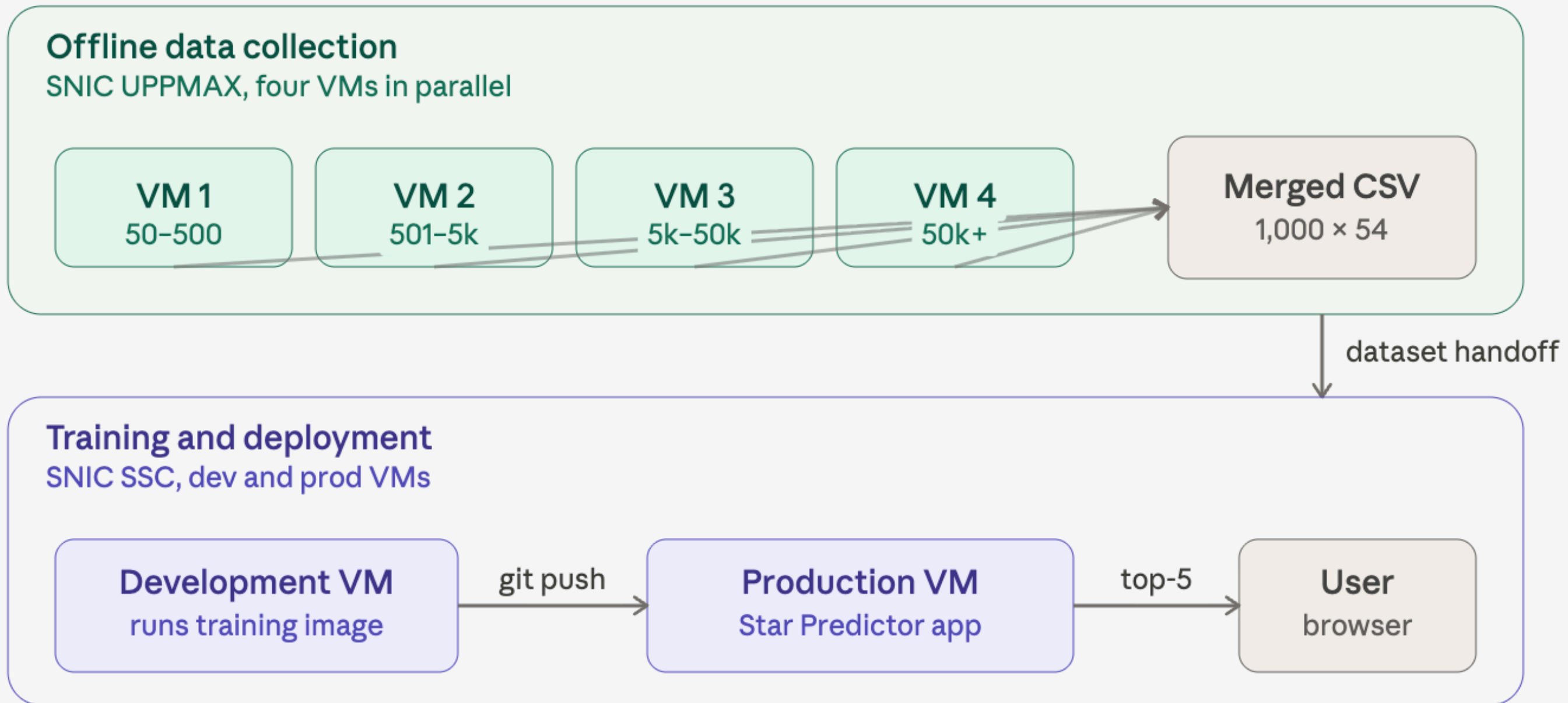
Goal: train a model that predicts star count from repo metadata, deploy it end-to-end, and evaluate the accuracy

Deliverables

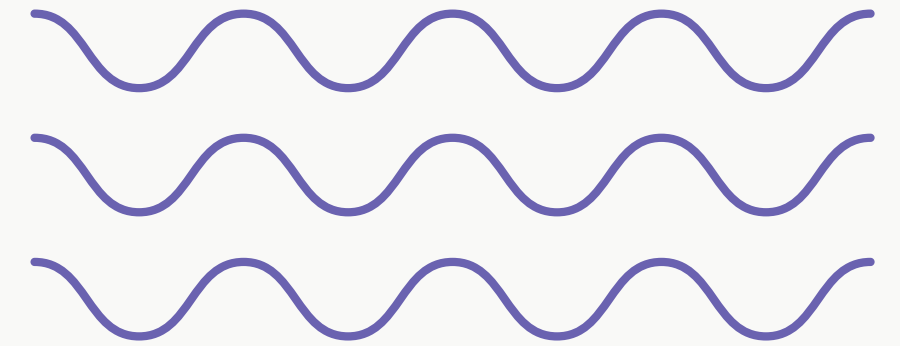
balanced dataset, model comparison, production app



System Architecture

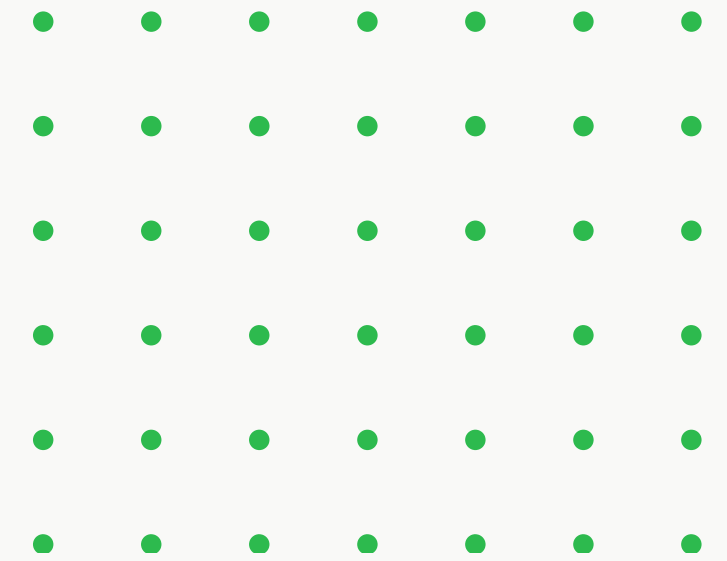


Data Collection



- Stratified uniform sampling across 4 log buckets (50–500, 501–5k, 5k–50k, 50k+)
- 250 repos per bucket → 1,000 total, spans 5 orders of magnitude
- ~13 API calls per repo (Search + Enrich phases)
- 4 VMs in parallel → 2–3 hours vs 8–12 sequential (~4× speedup)
- Robust to interruption: token rotation + checkpoint/resume

Feature Engineering & Model Training



Feature Engineering

54 raw columns → 44 input features

Engineered date-derived features

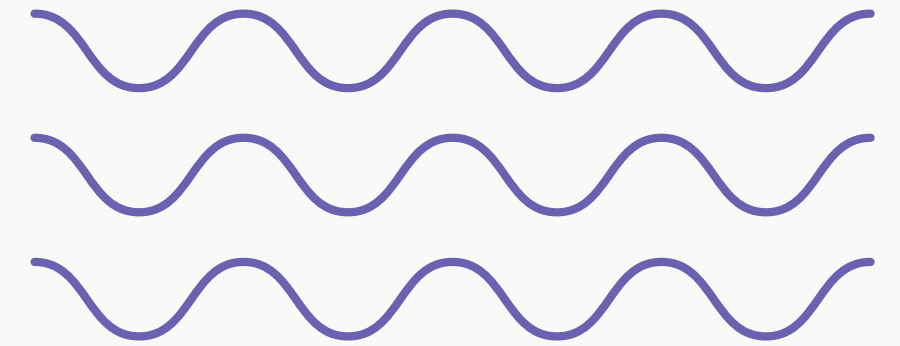
Target trained on $\log(1 + \text{stars})$ – five orders of magnitude

85/15 train-test split, 5-fold CV on training,

Models Trained

- Linear
- Ridge
- Random Forest
- Extra Trees
- Gradient Boosting
- Histogram Gradient Boosting
- XGBoost
- **MLP**

Deployment



- Two OpenStack VMs: dev (trains) and prod (serves)
- Two Docker images on Docker Hub: dev training image, prod app image
- Model promotion via Git post-receive hook: dev VM commits new artefacts, pushes to prod's bare repo
- Hook checks files out into mounted directory, restarts prod container
- Idempotent on artefact content; deployment defined entirely in Ansible

Star Predictor Application

Briefly discuss the key dates for the project.

 **DE-II Star Predictor**

[Production demo](#)

Rank GitHub repositories by predicted stars.

Show random top 5 ranking

Click to sample five test examples and display the predicted ranking.

Model Snapshot

5 random test examples ranked	R² test score 0.633
Random Forest selected ML model	44 features used

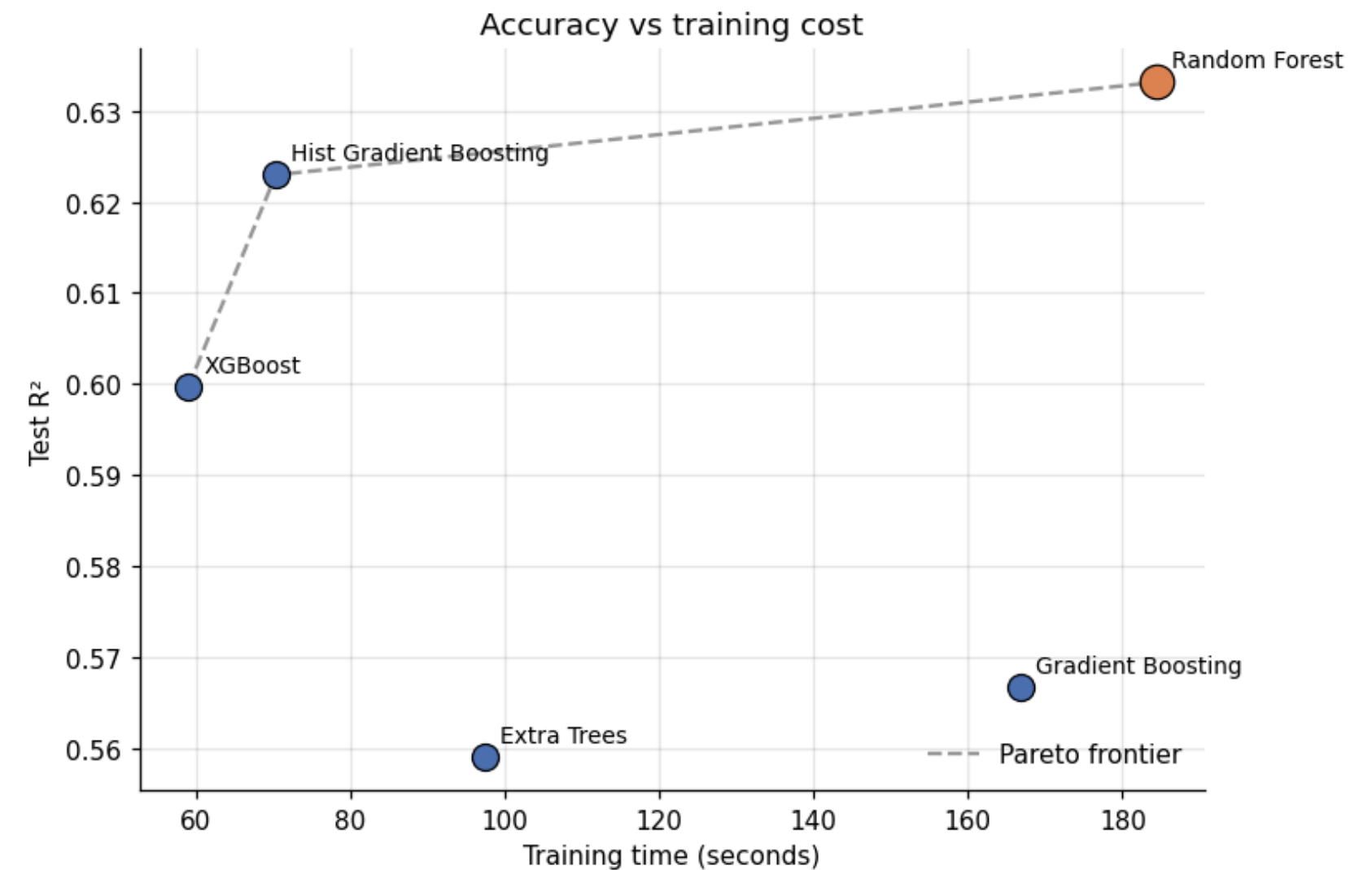
Star Predictor Application

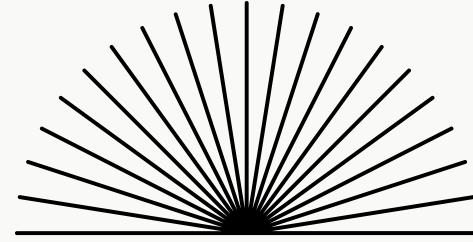
Predicted Ranking			5 sampled repositories
RANK	REPOSITORY	PREDICTED STARS	ACTUAL STARS
#1	Chalarangelo/30-seconds-of-code	112,716	127,885
#2	ansible/ansible	69,149	68,618
#3	ccfos/nightingale	17,771	13,029
#4	coder/coder	12,311	13,260
#5	TencentBlueKing/bk-lite	321	145

Results

- Random Forest selected as best model
- Linear baselines failed catastrophically
- Limitation: small dataset (1,000)

Model	Train R ²	Test R ²	Time (sec)
Random Forest	0.926	0.633	184.4
Hist GB	0.862	0.623	70.3
XGBoost	0.904	0.60	59.0
Gradient Boosting	0.856	0.567	166.9
Extra Trees	0.940	0.559	97.4





Thank you

Feedback and Questions are welcome

GROUP 7